

API Protocols

Integration is based on the following technologies:

- **REST** – discovery, game launching, game history, bonuses.
- **JSON-WSP** – seamless wallet API.

Required HTTP Headers

Header	Description
Content-Length	Size of the request body
Content-Type	<code>application/json</code>

Endpoints and Base API URL

Example Base API URL:

```
https://game-aggregator.com/api/v1
```



Example endpoint:

```
/seamless/bet
```



Final URL:

```
https://game-aggregator.com/api/v1/seamless/bet
```



Timeouts

The aggregator enforces response timeouts per game type. If the operator does not respond within the timeout window, the aggregator treats the request as failed and takes the corresponding action.

Game Type	Timeout	Examples
Crash games	1 second	Crash, Aviator-style games
All others	5 seconds	Slots, table games, live casino

Aggregator behavior on timeout:

Operation	On timeout	Why
Bet	Sends rollback	Unclear if money was deducted — protect the player.
Win	Retries until success	Player already lost money and must be paid.
Rollback	Retries until success	Consistency must be restored.
Balance	Retries once , then uses cached value	Non-transactional, low risk.

**Operator responsibility**

Crash games require sub-second response times because the game state changes continuously (multiplier is ticking). A slow wallet response will break the player experience. Operators supporting crash games must optimize their wallet endpoints accordingly.

Capacity

The aggregator does **not** apply rate limiting — request volume is driven by real player activity. The operator must provision sufficient capacity to handle their expected concurrent player load. Expected throughput should be discussed with the account manager during the integration phase.

Amount Precision

When building the **signature string** (X-Sign), all monetary amounts (`amount` , `balance`) must be formatted with a fixed number of decimal places matching the currency's **precision scale**. The JSON body itself may contain any valid numeric representation (e.g. `1` , `1.0` , `1.00` are all valid JSON), but the signing string must always use the normalized format.

Currency	Precision Scale	Example
USD	2	<code>1.00</code> , <code>10.50</code>
EUR	2	<code>1.00</code> , <code>10.50</code>
GEL	2	<code>1.00</code> , <code>10.50</code>

Other currencies

Precision scale for additional supported currencies will be provided by the account manager during the integration phase.

Formatting matters for signature calculation

When building the X-Sign signature string, amounts **must** be normalized to the full precision scale. For example, for USD: if the JSON body contains `"amount": 1` , the signing string must use `amount=1.00` . See [Security – Amount formatting in signatures](#) for details.